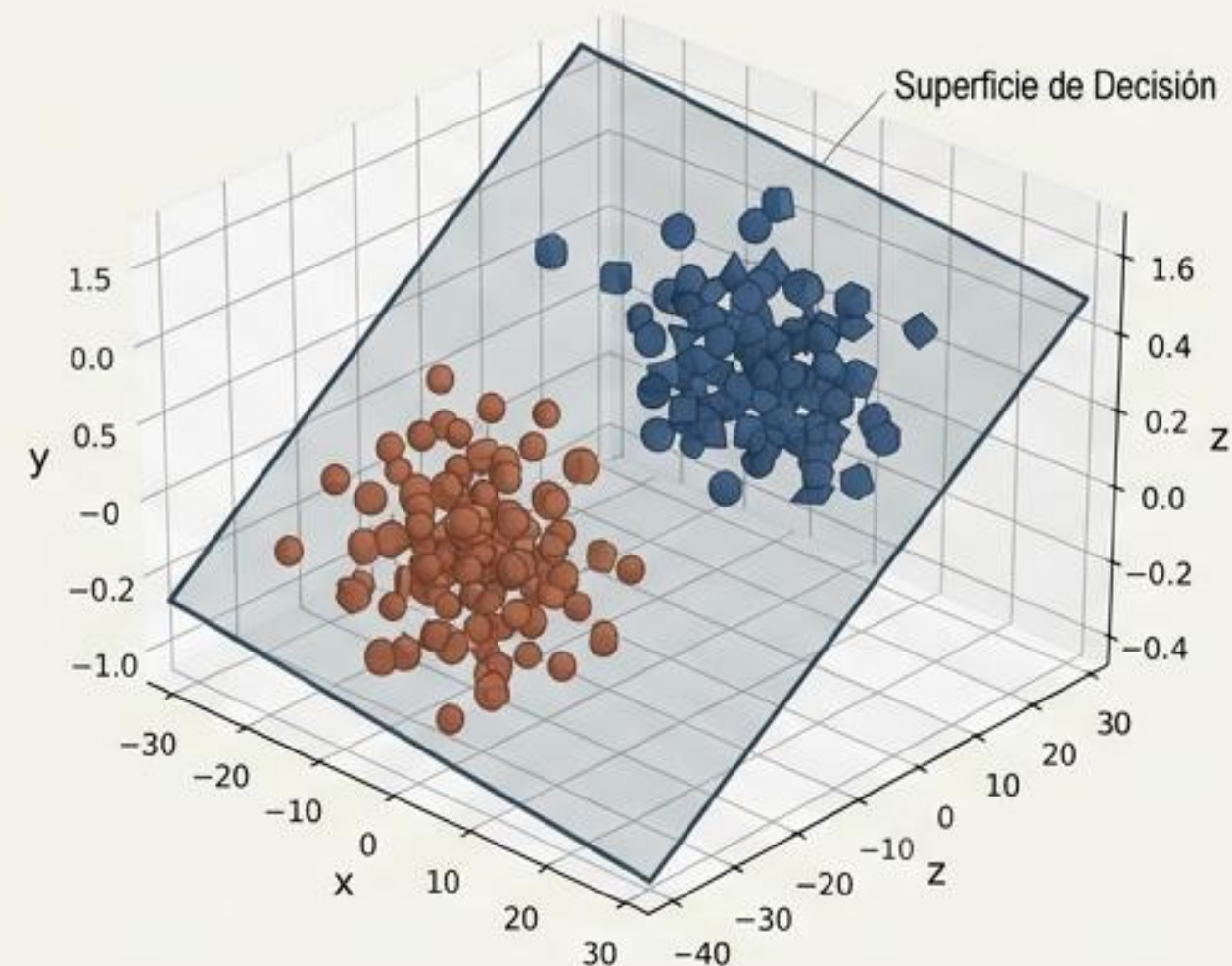


Modelos de Clasificación: Fundamentos, Geometría y Aplicación

Ingeniería del Conocimiento (ISO56B) — Universidad Nacional del Centro del Perú (UNCP)

Docente: Msc. Jaime Antonio Huaytalla Pariona

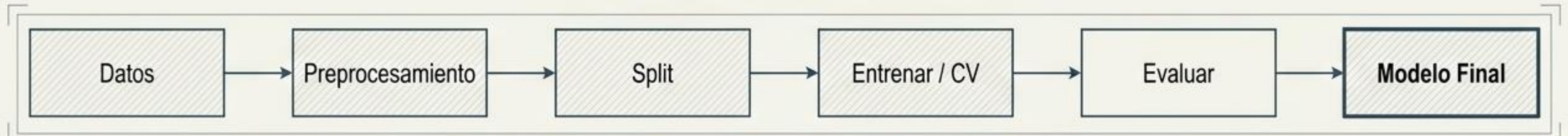


$$f(x, y, z) = w_1x + w_2y + w_3z + b = 0$$

Ecuación del Plano Separador

Visualización Geométrica de un Modelo de Clasificación Lineal en 3D

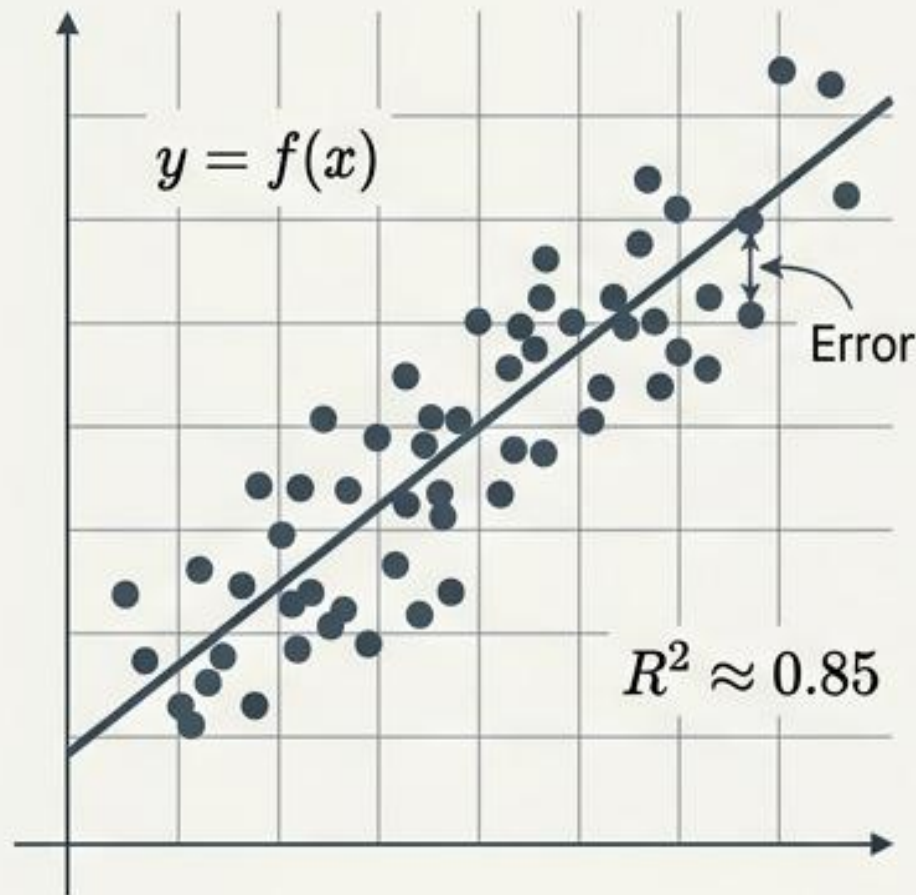
La superficie plana divide el espacio vectorial, aislando los puntos de Clase 0 (Terracotta) de los de Clase 1 (Sapphire). La geometría define la frontera de decisión.



Flujo de Trabajo del Modelado de Clasificación: Desde los Datos Crudos hasta el Modelo Implementable.

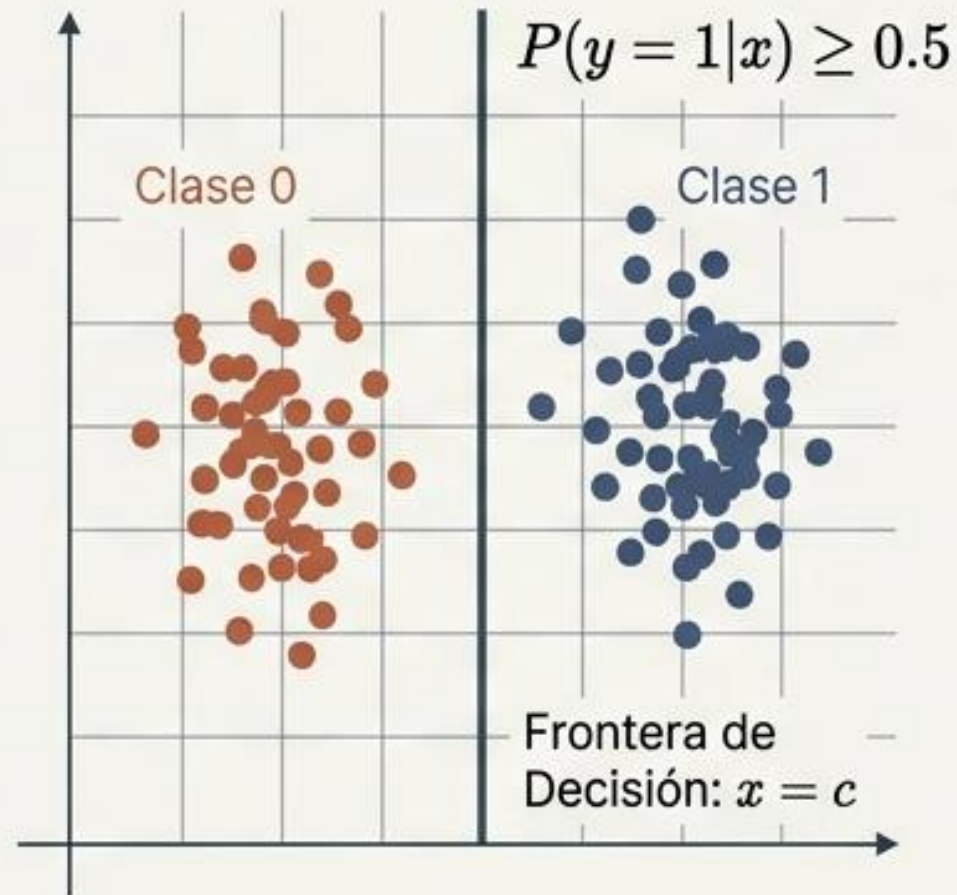
El Paisaje Espacial de la Clasificación

Regresión
("¿Cuánto?")



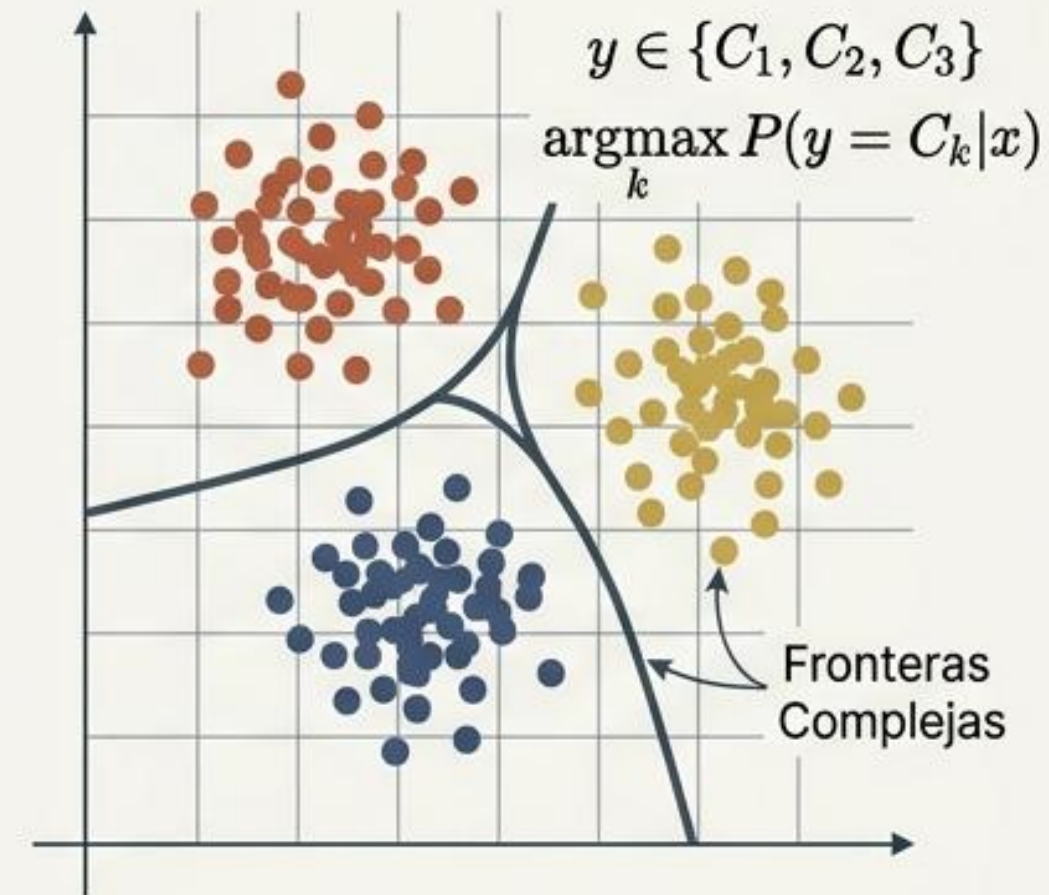
Predicción de valores continuos.

Clasificación Binaria
("¿Qué categoría?")



Predicción de etiquetas discretas (0 o 1).

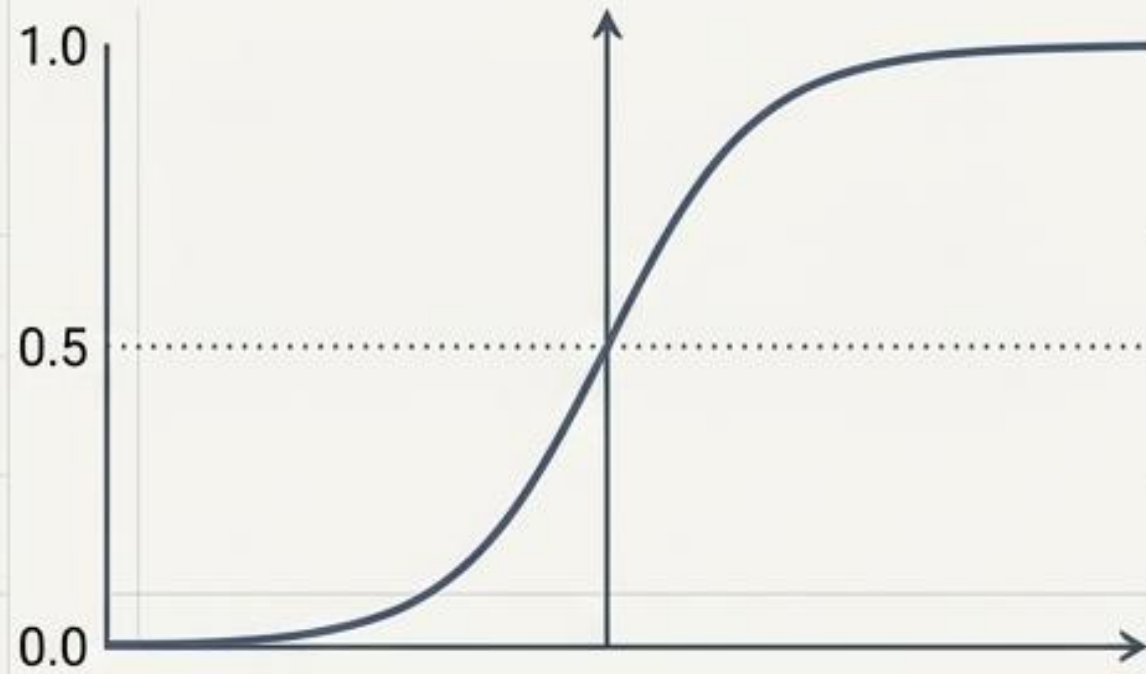
Clasificación Multiclase
y Multietiqueta



Múltiples fronteras categóricas mutuamente excluyentes o simultáneas.

A diferencia de la regresión, la clasificación aborda la pertenencia a una categoría, transformando features de entrada en distribuciones de probabilidad discretas.

Regresión Logística: El Baseline Probabilístico

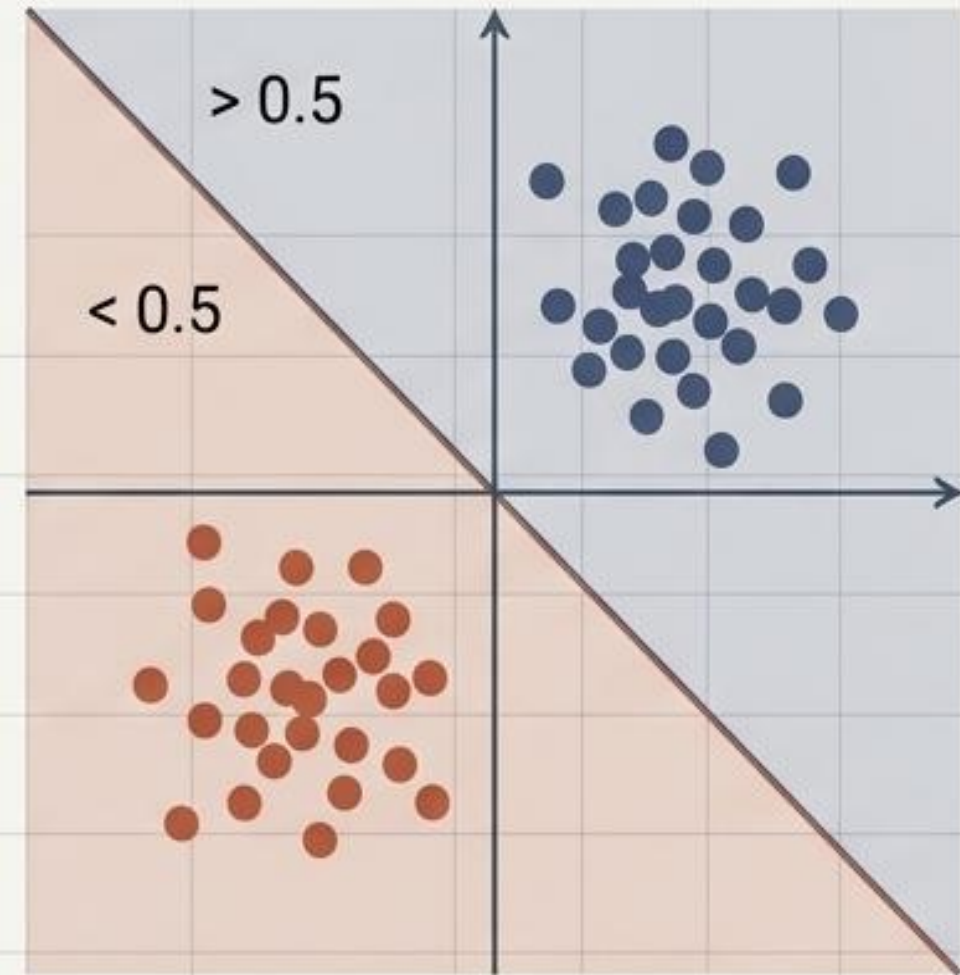


Función Hipótesis:

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Extensión Multiclase (Softmax):

$$P(y = k|x) = \frac{e^{z_k}}{\sum e^{z_j}}$$



Frontera Lineal:

Predice Clase 1 si $h_{\theta}(x) \geq 0.5$ (donde $\theta^T x \geq 0$).

Mapea una combinación lineal de features a una probabilidad entre 0 y 1.

Optimización, Regularización y Práctica

Fundamento Matemático

Función de Costo (Log-Loss / Cross-Entropy):

$$J(\theta) = -\frac{1}{m} \sum_i^n [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Regla de Actualización (Gradient Descent):

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_j^n (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

Regularización

- L2 (Ridge): Reduce la magnitud de los coeficientes.
- L1 (Lasso): Selecciona features (lleva coeficientes a cero).
- El parámetro C en Scikit-Learn es el inverso de la fuerza de regularización.

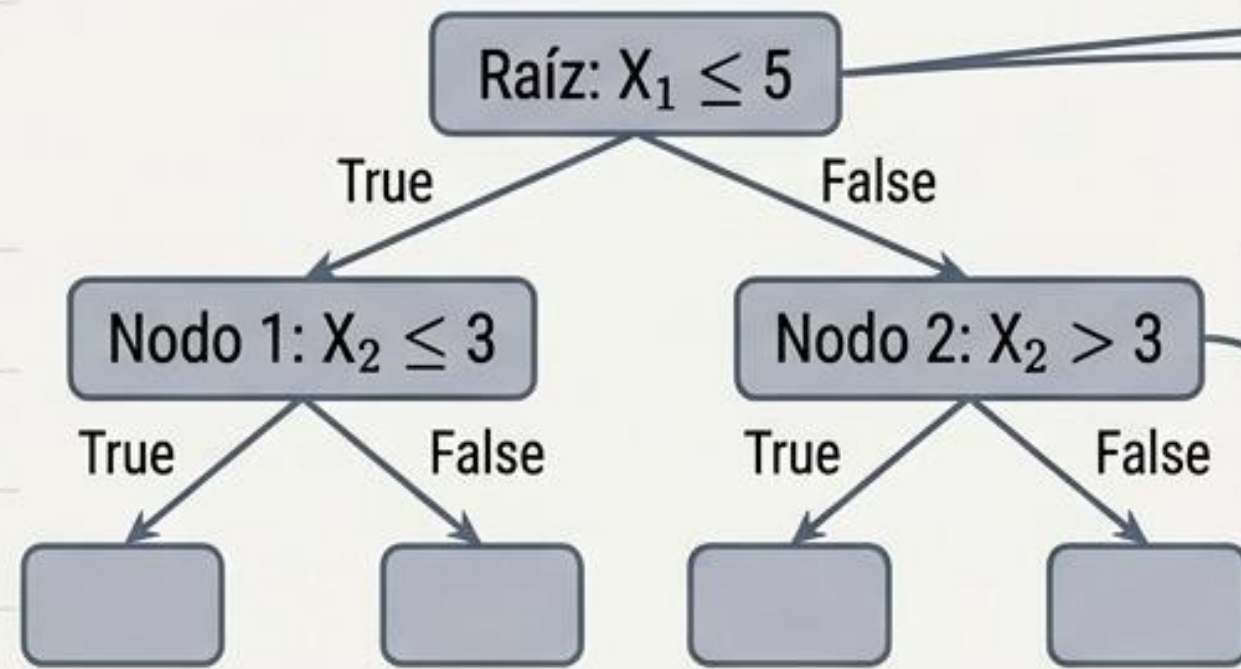
Implementación (Scikit-Learn)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(C=1.0, penalty='l2'))
])

pipe.fit(X_train, y_train)
y_prob = pipe.predict_proba(X_test)
```

Árboles de Decisión: Particionando el Espacio

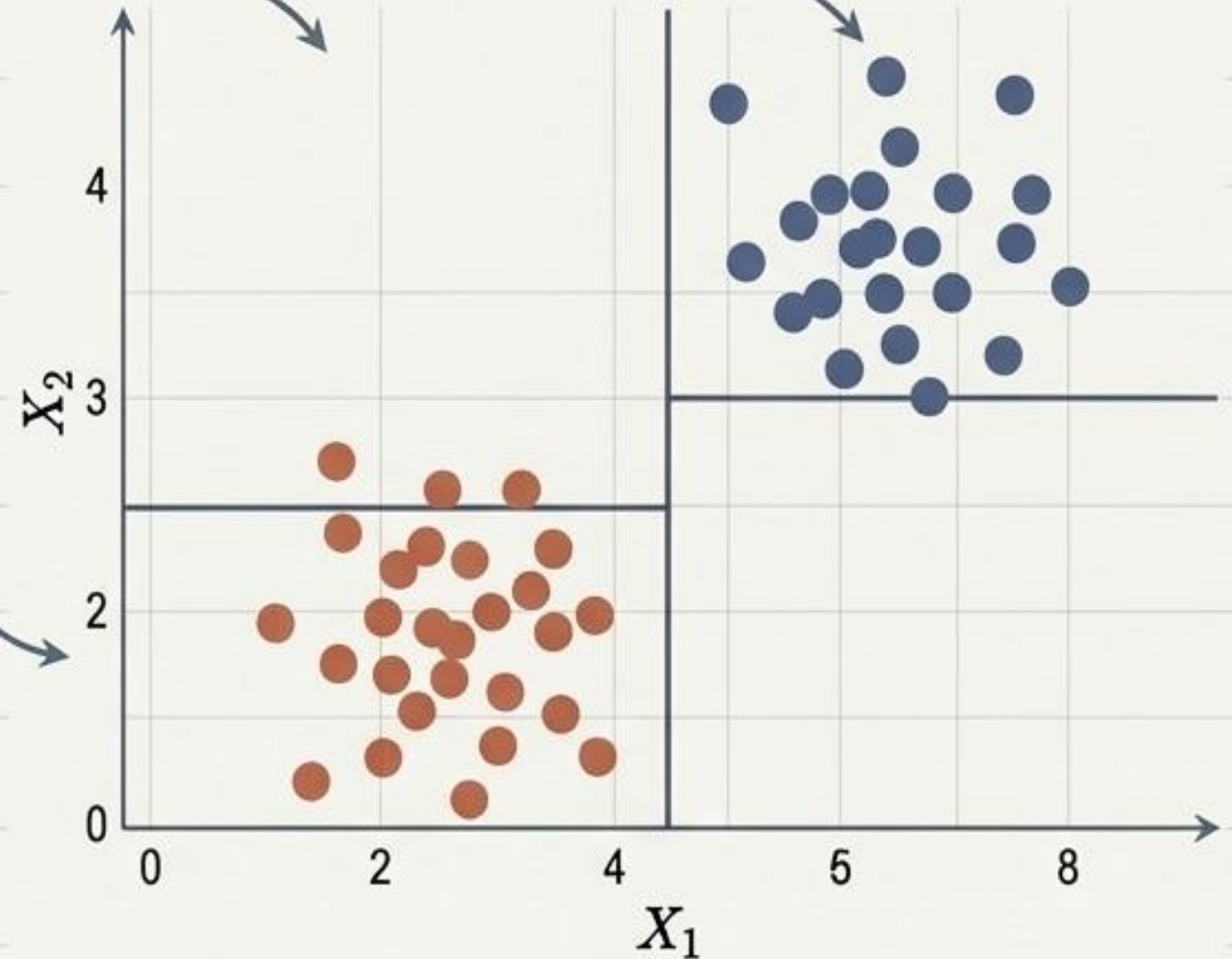


Impureza de Gini:

$$G(t) = 1 - \sum p_k^2$$

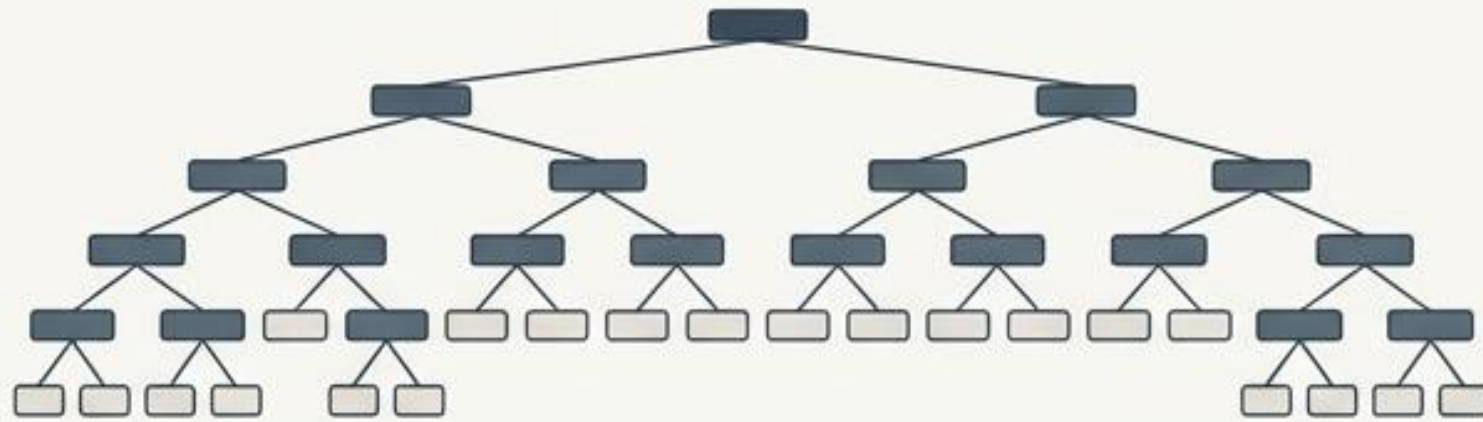
Entropía:

$$H(t) = - \sum p_k \log_2(p_k)$$

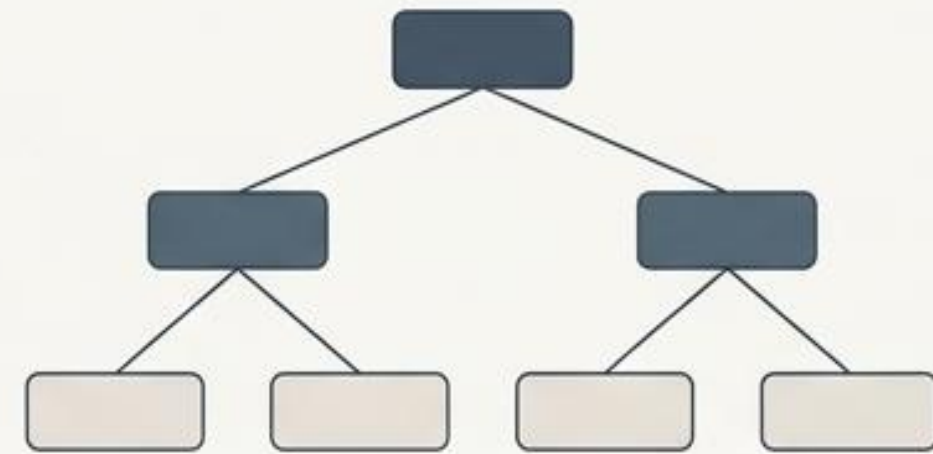


Modelos no paramétricos (algoritmo CART) que particionan recursivamente el espacio mediante reglas ortogonales (if-then-else). El árbol selecciona la partición que maximiza la Ganancia de Información, buscando los nodos hijos más puros.

Controlando el Caos: Poda e Implementación



Sobreajuste (Sin Poda)



Generalización (Con Poda)

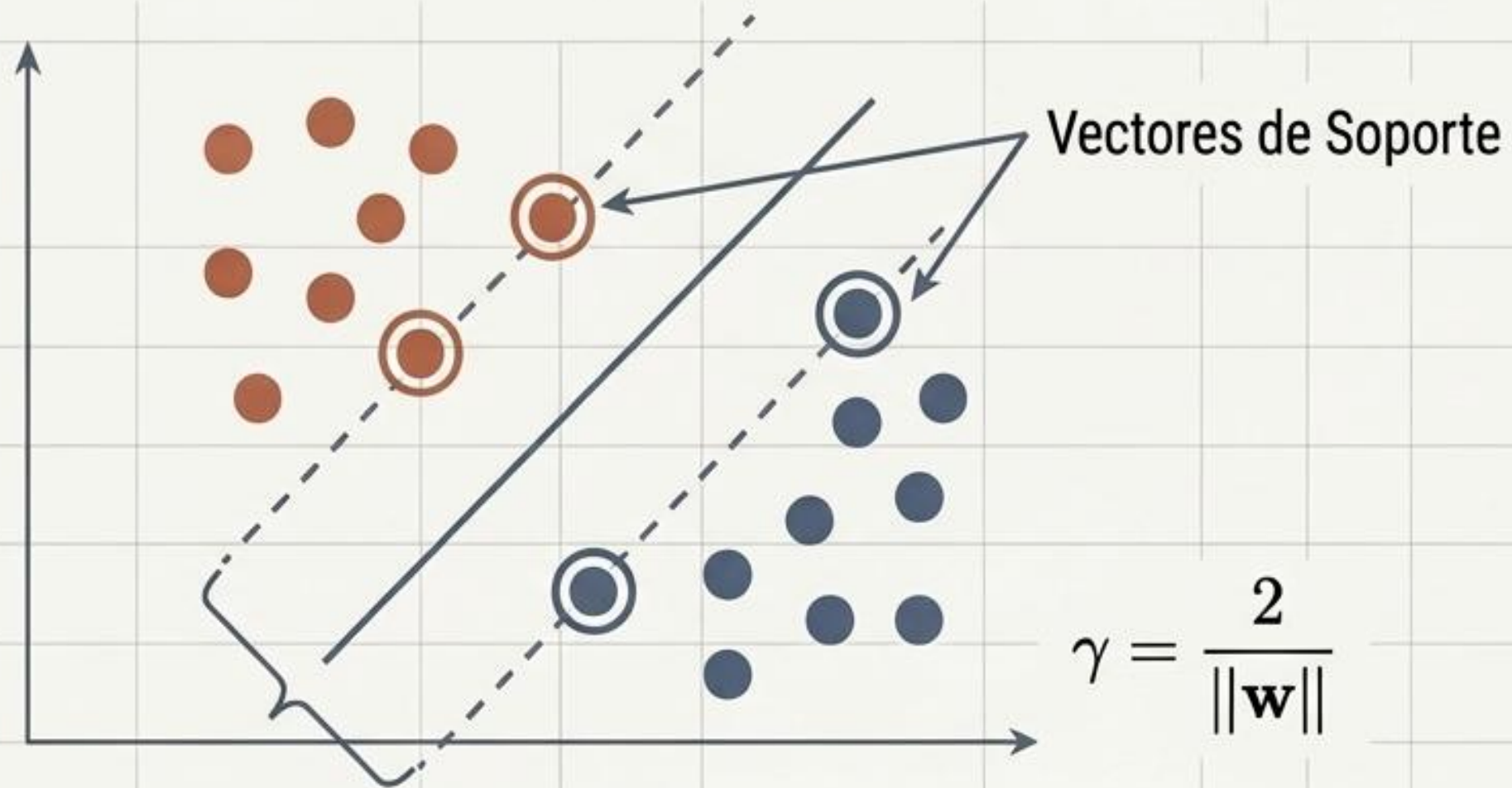
Pre-poda	Post-poda
• max_depth • min_samples_split • min_samples_leaf	• ccp_alpha (Cost Complexity Pruning)

Importancia de Features (**clf.feature_importances_**) mide la reducción total de impureza aportada por cada variable.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

clf = DecisionTreeClassifier(
    max_depth=5, min_samples_leaf=10, criterion='gini'
)
clf.fit(X_train, y_train)
importances = clf.feature_importances_
```

SVM: Maximizando el Margen Geométrico



Objetivo Matemático

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum \xi_i$$

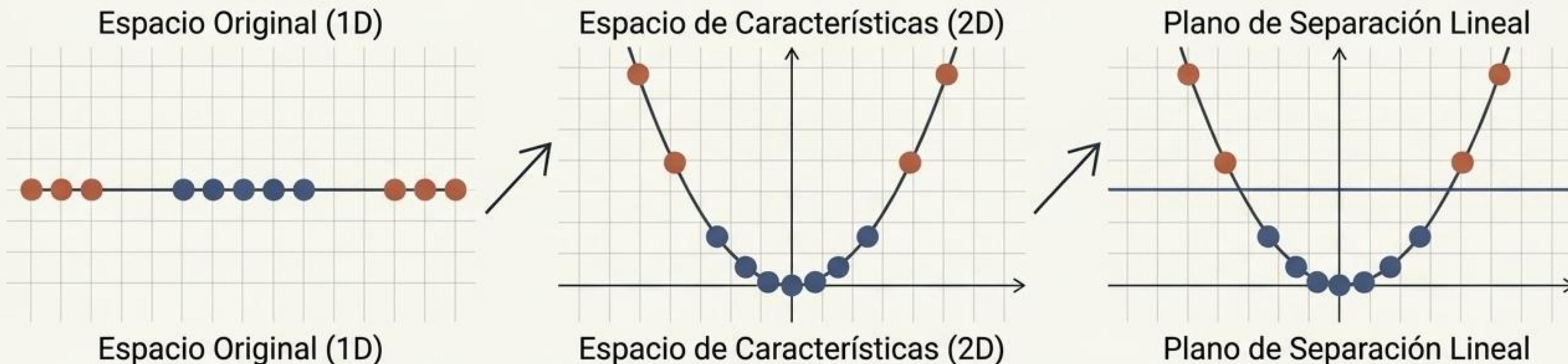
Soft Margin

Las variables de holgura (ξ_i) permiten tolerar errores en datos no linealmente separables.

Parámetro C

- Un valor C grande estrecha el margen (mayor ajuste a datos).
- Un valor C pequeño amplía el margen (mayor regularización).

El Truco del Kernel y la No Linealidad



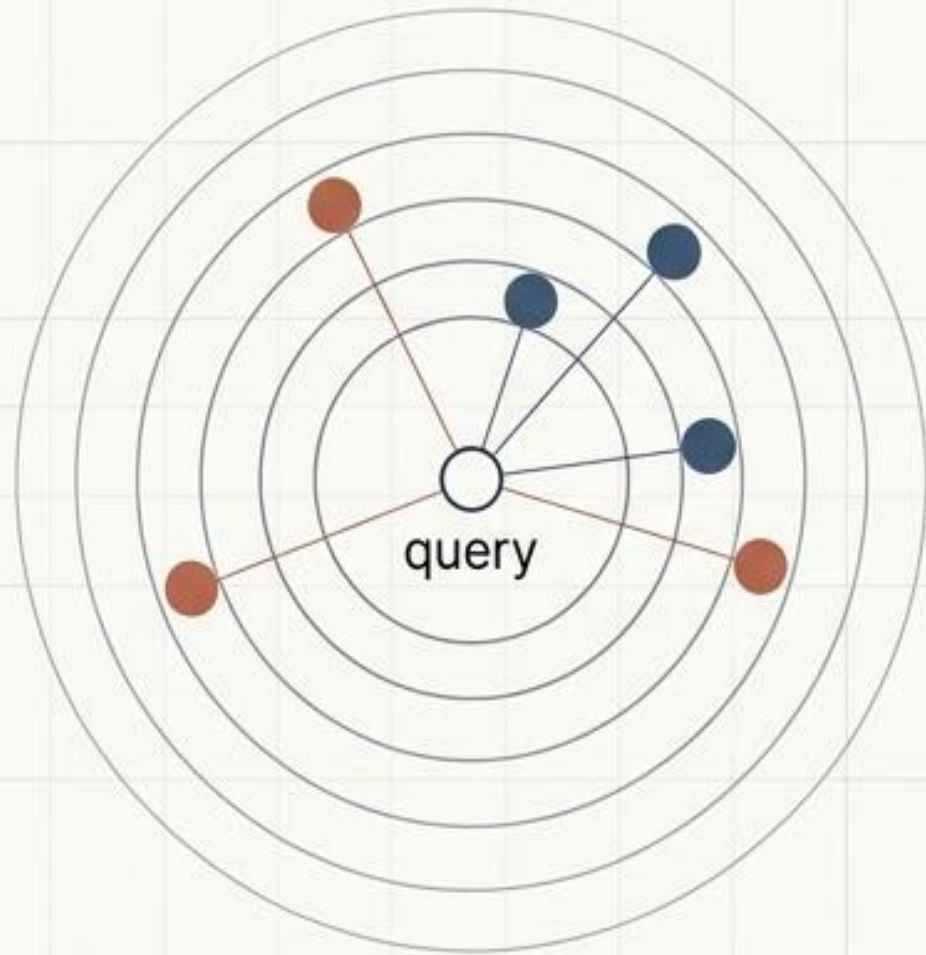
Kernels Comunes:

Kernel	Fórmula y Uso
• Lineal	$K(x,z) = x^T z$ (Alta dimensionalidad)
• RBF (Gaussiano)	$K(x,z) = \exp(-\gamma x-z ^2)$ (Caso general no lineal)
• Polinomial	$K(x,z) = (\gamma x^T z + r)^d$

Nota: γ (Gamma) controla el radio de influencia. γ grande = frontera compleja.

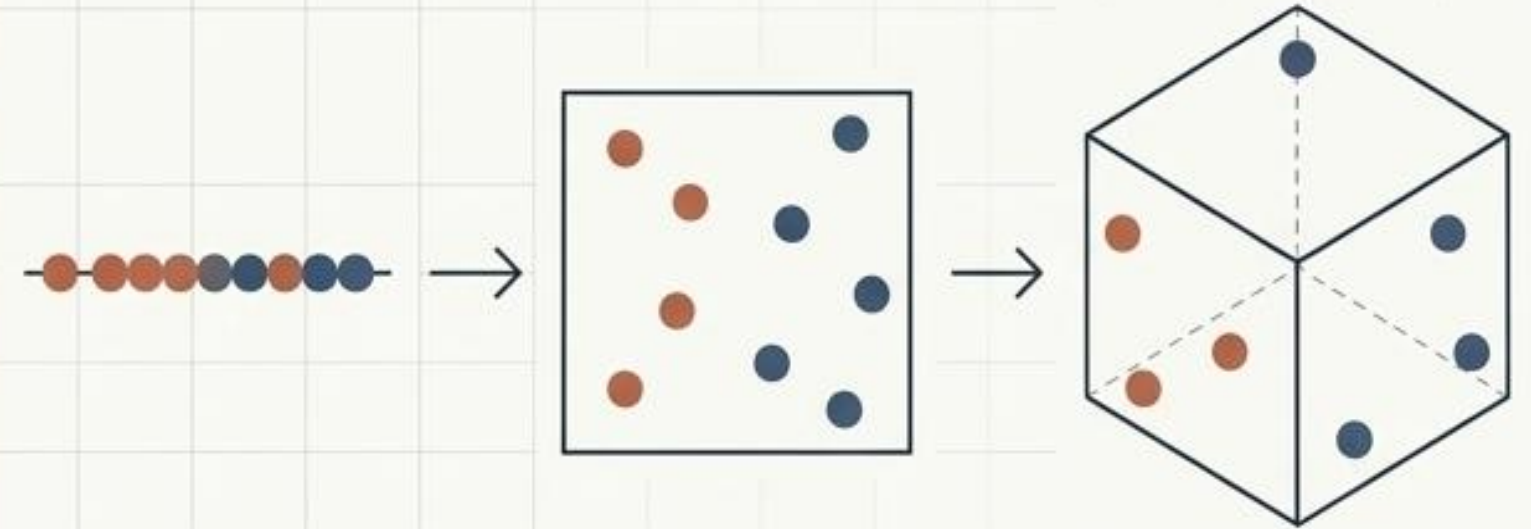
```
from sklearn.svm import SVC
# SVM es altamente sensible a la escala
pipe_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', SVC(kernel='rbf', C=1.0, gamma='scale', probability=True))
])
```

K-Nearest Neighbors: Aprendizaje Basado en Instancias



Distancias: Euclidiana (L2), Manhattan (L1), Minkowski (Lp), Coseno.

Elegir $K = \sqrt{n}$. Preferir K impar.

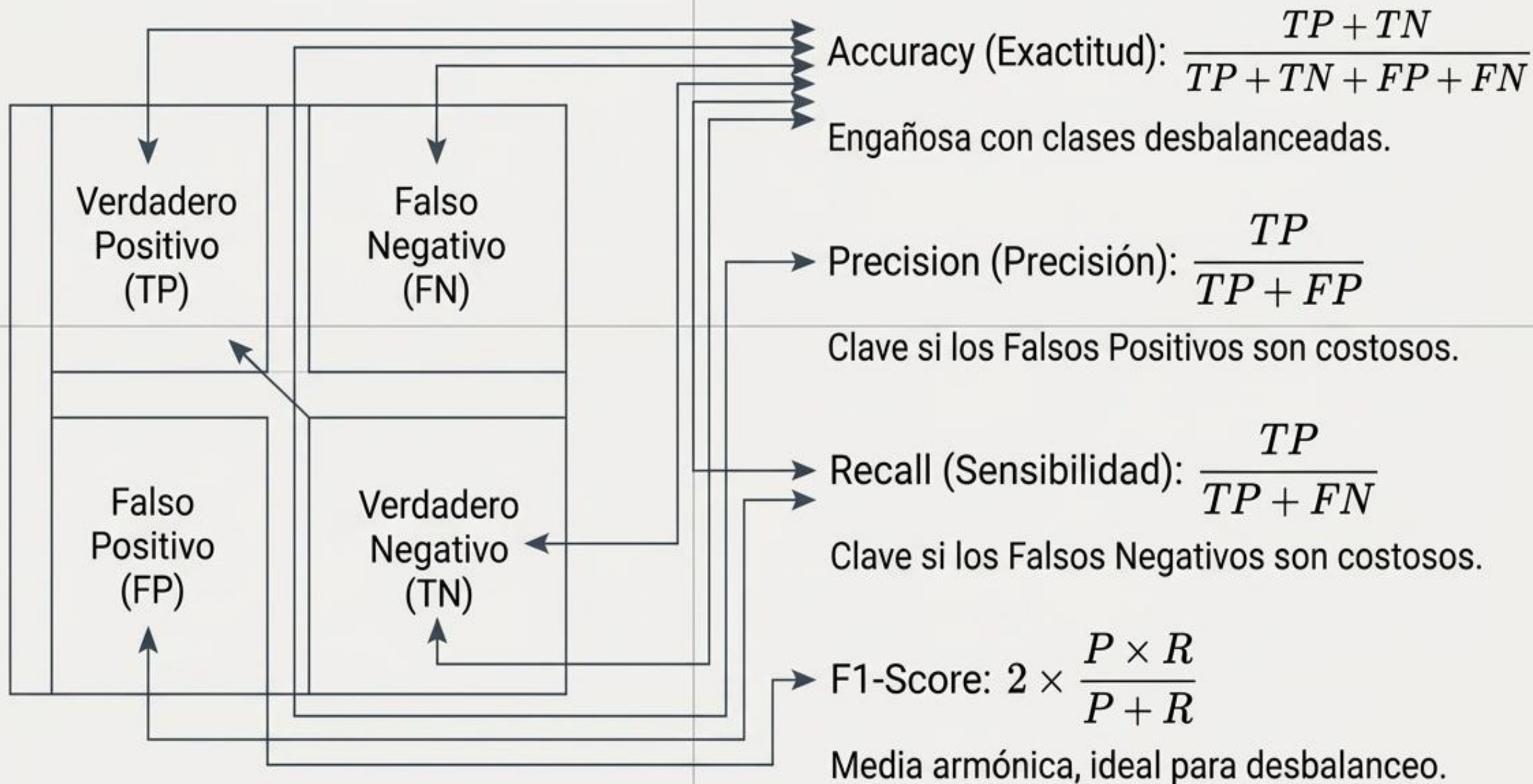


Maldición de la Dimensionalidad:

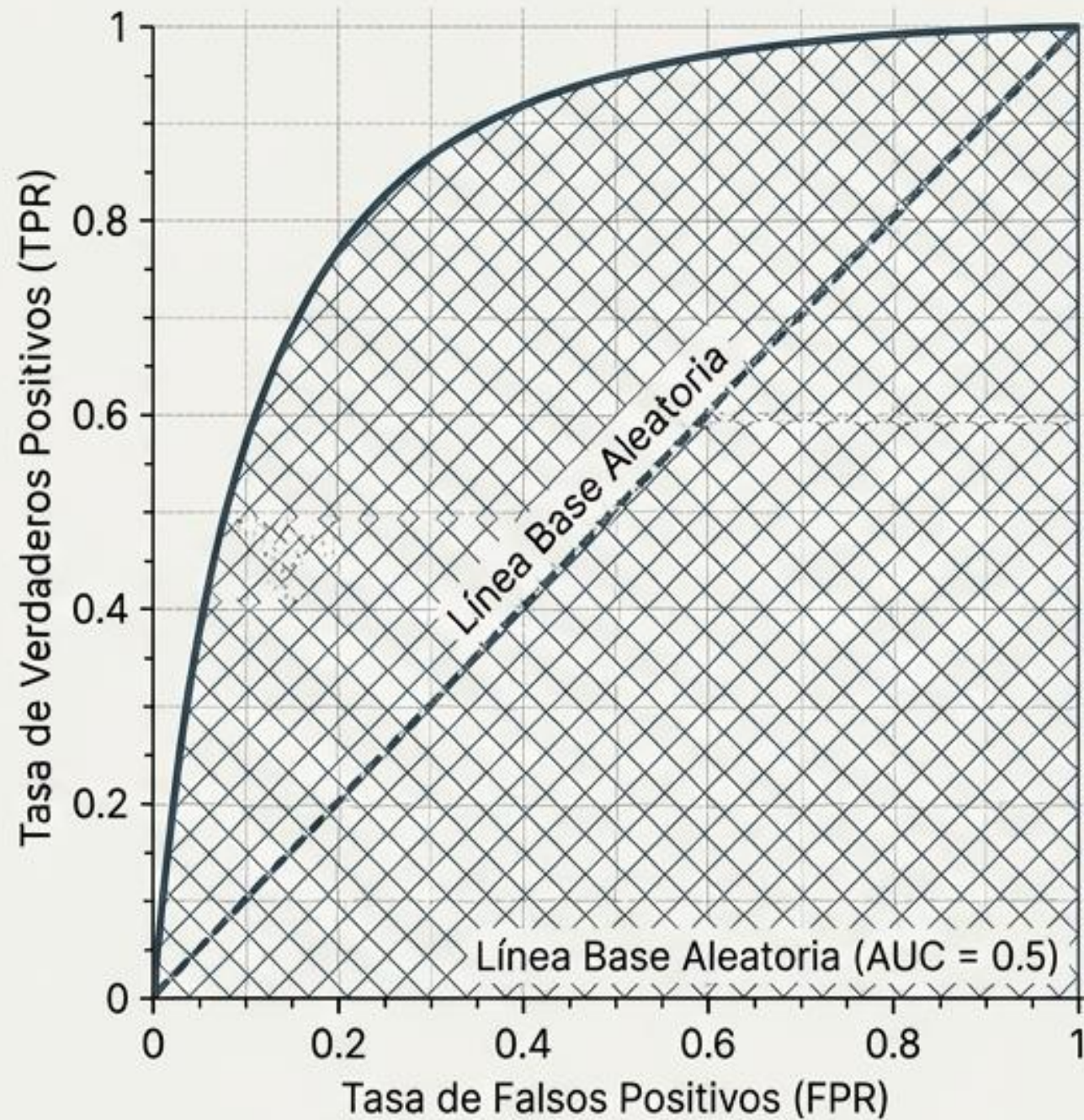
A medida que crecen las features, las distancias entre puntos se vuelven uniformes, degradando el rendimiento. Requiere estandarización obligatoria.

```
from sklearn.neighbors import KNeighborsClassifier  
knn = KNeighborsClassifier(n_neighbors=5, weights='distance', metric='minkowski', p=2)
```

Midiendo el Éxito: La Matriz de Confusión



Métricas Avanzadas: Curva ROC, AUC y Multiclase



AUC = 1.0 es perfecto, 0.5 es aleatorio.
Métrica independiente del umbral.

Estrategias Multiclase:

- Macro: Promedio no ponderado (trata todas las clases por igual).
- Weighted: Promedio ponderado por la frecuencia de cada clase.
- Micro: Métricas globales (equivale a accuracy).

```
from sklearn.metrics import classification_report, f1_score
print(classification_report(y_test, y_pred))
f1_macro = f1_score(y_test, y_pred, average='macro')
```

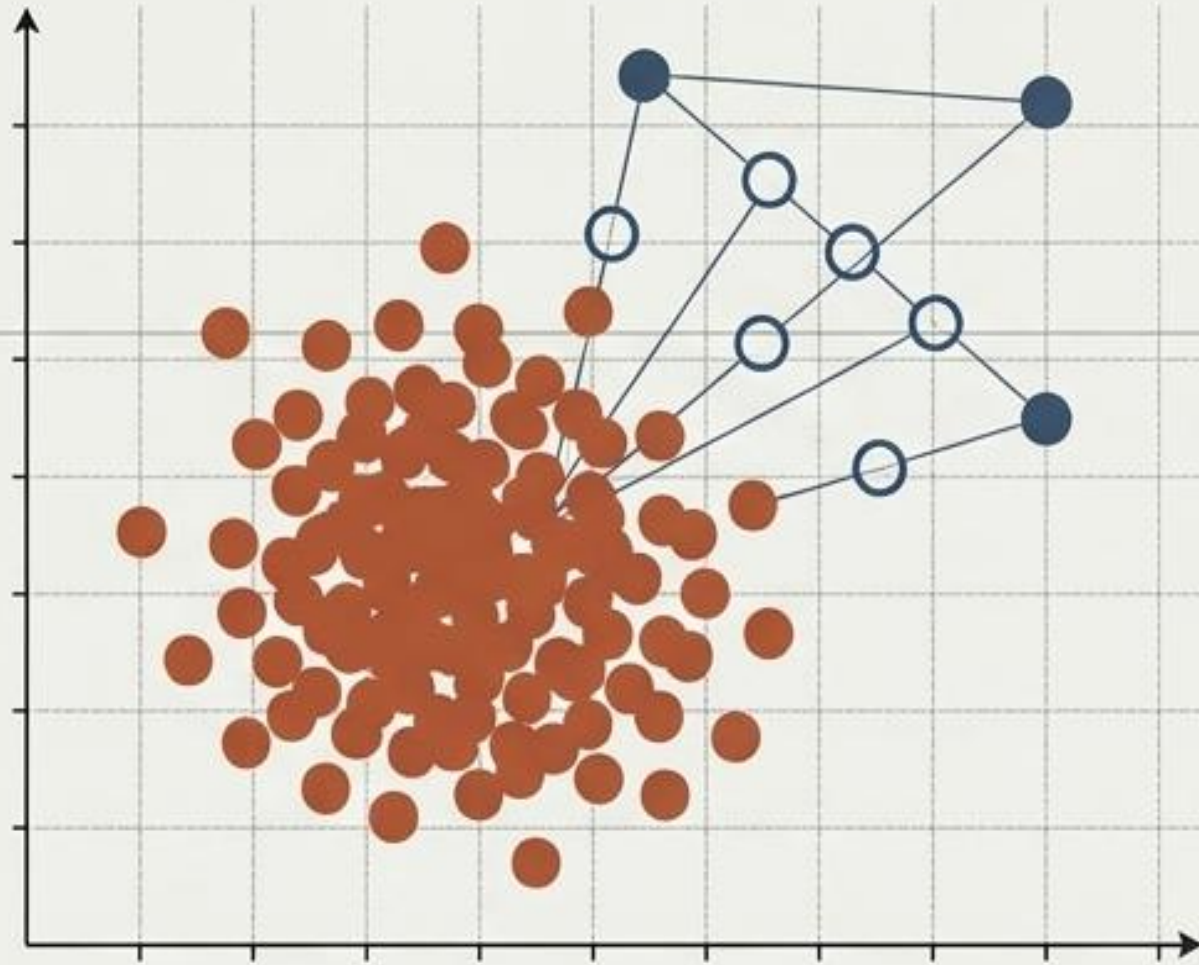
La Matriz Diagnóstica: Selección de Modelos

Criterio	Regresión Logística	Árbol de Decisión	SVM	KNN
Interpretabilidad	Alta (coeficientes)	Muy Alta (visual)	Baja (Kernel)	Media
Escalado Requerido	[Sí] Para regularización	[No] Ortogonal	[Sí] Obligatorio	[Sí] Obligatorio
Velocidad Predicción	Muy Rápida	Muy Rápida	Rápida	Lenta $O(n \cdot d)$
Alta Dimensionalidad	Bueno	Moderado	Excelente	Malo (Maldición)
Probabilidades	Nativas calibradas	Nativas escalonadas	Requiere ajuste	Por proporción

El Desafío del Mundo Real: Desbalanceo de Clases

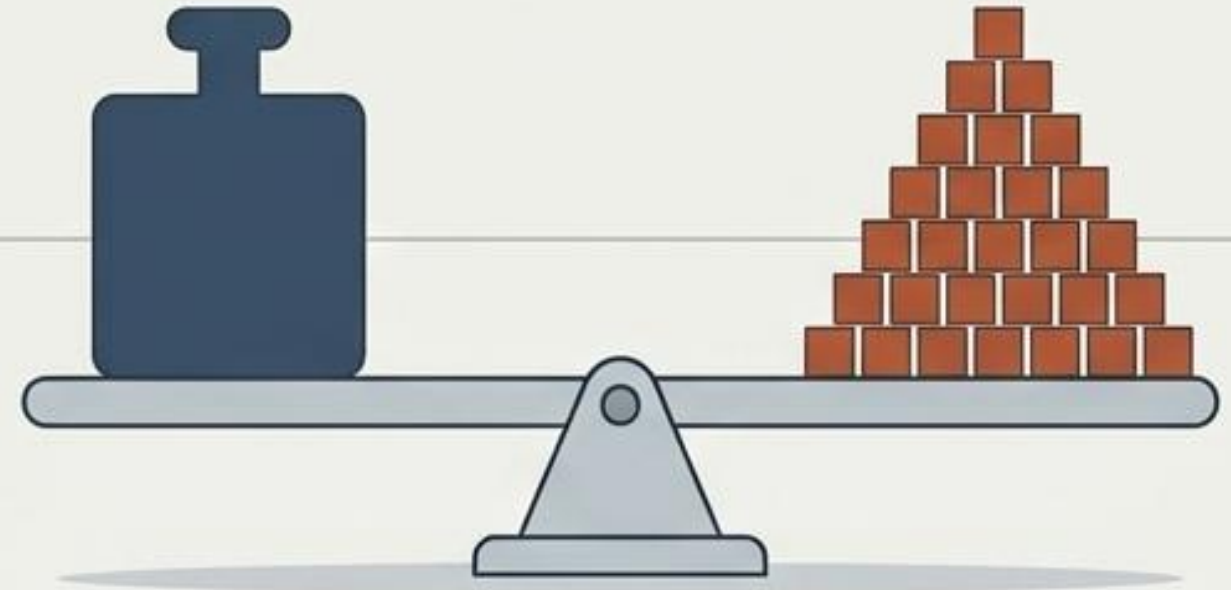
Accuracy es inútil frente a clases raras (ej. 99% vs 1%).

A Nivel de Datos - SMOTE



Estrategias de Datos: Undersampling de la mayoría, o Oversampling sintético con SMOTE interpolando vectores.

A Nivel Algorítmico



Estrategias Algorítmicas: Penalizar errores en la clase minoritaria ponderando la función de costo.

```
clf = LogisticRegression(class_weight='balanced')
```

Validación Robusta: Stratified K-Fold

Dataset Total



Una sola partición Train/Test genera estimaciones sesgadas.

Stratified K-Fold garantiza que la proporción de clases se mantenga constante en cada subconjunto.

Obligatorio en problemas de clasificación.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(clf, X, y, cv=skf, scoring='f1_macro')
```

Datasets de Referencia y Laboratorio Práctico

Iris (Flores)



Clases: 3 | **Muestras:** 150 | **Features:** 4

Baseline ideal.

Breast Cancer (Diagnóstico)



Clases: 2 | **Muestras:** 569 | **Features:** 30

Prueba de recall.

Digits (Imágenes MNIST)



Clases: 10 | **Muestras:** 1797 | **Features:** 64

Multiclase, alta dimensionalidad.

Titanic (Supervivencia)



Clases: 2 | **Muestras:** 891 | **Features:** ~10

Manejo de variables categóricas.

Próximos pasos: Transición del diseño matemático a la implementación con Scikit-Learn en la sesión de laboratorio.